실습 7차시

CLAUDE.md와 프로젝트 설정

학습 목표

- ✓ CLAUDE.md 파일에 프로젝트 규칙과 스타일 가이드를 작성할 수 있어요
- ✓ Claude Code에서 프로젝트 설정을 구성하고 동작을 확인할 수 있어요
- ✓ CLAUDE.md 유/무에 따른 Claude 응답 일관성 차이를 비교할 수 있어요

? 핵심 질문

같은 AI에게 같은 질문을 해도 매번 다른 답이 나와요. 프로젝트마다 원하는 스타일과 규칙이 있는데, 어떻게 하면 AI가 이걸 기억하고 따르게 할 수 있을까요?

0. 오늘 만들 결과물

이번 실습이 끝나면 여러분의 프로젝트 폴더에 `CLAUDE.md` 파일이 생겨요. 이 파일 하나가 Claude Code의 행동 방식을 완전히 바꿔요. 마치 신입 직원에게 회사 매뉴얼을 건네는 것과 같아요. 매뉴얼 없이 일하면 매번 물어봐야 하지만, 매뉴얼이 있으면 스스로 판단하고 움직이거든요.

배경지식: 컨텍스트 엔지니어링과 CLAUDE.md

이론편에서 컨텍스트 엔지니어링을 배웠어요. 프롬프트만으로는 한계가 있고, AI에게 더 풍부한 맥락을 제공해야 좋은 결과가 나온다는 내용이었죠. CLAUDE.md는 이 원리를 파일 하나로 구현한 거예요.

CLAUDE.md는 Claude Code가 프로젝트를 열 때 자동으로 읽는 설정 파일이에요. 여기에 적힌 내용이 모든 대화의 시스템 프롬프트처럼 작동해요. 빌드 명령어, 코딩 스타일, 프로젝트 구조 같은 정보를 한번 적어두면, 매번 설명하지 않아도 Claude가 알아서 따라요.

준비물

- Claude Code가 설치된 환경 (터미널에서 `claude` 명령어 실행 가능)
- 빈 프로젝트 폴더 1개 (아무 이름이나 괜찮아요)
- 텍스트 에디터 (VS Code 권장)

1. Claude Code 설치 확인

1 Claude Code가 잘 설치되어 있는지 확인해요

먼저 터미널을 열어보세요. macOS라면 Terminal, Windows라면 PowerShell이나 WSL 터미널을 사용해요.

아래 명령어를 입력해보세요.

```
claude --version
```

버전 번호가 출력되면 설치가 완료된 상태예요. 만약 “command not found”가 나온다면 아직 설치가 안 된 거예요.

직접 해보기

설치가 안 되어 있다면 아래 명령어로 설치해보세요.

```
npm install -g @anthropic-ai/claude-code
```

Node.js가 설치되어 있어야 해요. Node.js가 없다면 https://nodejs.org에서 먼저 설치하세요. LTS 버전을 권장해요.

설치 후 다시 `claude --version`을 실행해서 버전이 출력되는지 확인해보세요.

[스크린샷: 터미널에서 claude –version을 실행하고 버전 번호가 출력되는 화면]

설치가 확인되었으면 Claude Code를 한번 실행해볼게요. 터미널에서 아무 폴더나 들어가서 `claude`를 입력하면 대화형 인터페이스가 열려요.

```
cd ~/Desktop
claude
```

Claude Code가 실행되면 프롬프트가 바뀌면서 대화를 시작할 수 있어요. 간단한 질문을 해보세요.

```
> 안녕, 지금 어떤 폴더에서 실행되고 있는지 알려줘
```

Claude가 현재 디렉토리 경로를 알려줄 거예요. 이것이 Claude Code의 핵심이에요. Claude는 자기가 어떤 폴더에서 실행되었는지를 알고 있고, 그 폴더의 파일을 읽고 수정할 수 있어요.

[스크린샷: Claude Code 대화 인터페이스에서 현재 폴더를 물어보고 답변을 받는 화면]

> Claude Code는 단순한 채팅이 아니에요. 실제 파일 시스템에 접근할 수 있는 AI 에이전트예요. 이 점이 웹 기반 Claude와 가장 큰 차이예요.

`/exit`을 입력해서 일단 나오세요. 다음 단계에서 본격적으로 프로젝트를 만들어볼 거예요.

2. 빈 프로젝트에서 시작

2 실습용 프로젝트 폴더를 만들어요

먼저 실습용 프로젝트 폴더를 만들어볼게요. 이 프로젝트는 간단한 할 일 관리 앱을 가정할 거예요. 실제로 코드를 작성하지 않아도 괜찮아요. CLAUDE.md의 효과를 확인하는 것이 목표니까요.

```
mkdir ~/practice-todo-app
cd ~/practice-todo-app
```

프로젝트의 기본 구조를 잡아볼게요. 아래 명령어로 폴더와 파일을 만들어보세요.

```
mkdir src tests docs
touch src/main.py src/models.py src/utils.py
touch tests/test_main.py
touch README.md
```

이제 이 폴더에서 Claude Code를 실행해보세요.

```
claude
```

CLAUDE.md가 없는 상태에서 먼저 Claude에게 작업을 시켜볼 거예요. 이렇게 입력해보세요.

```
> src/main.py에 할 일 목록을 관리하는 기본 코드를 작성해줘
```

[스크린샷: CLAUDE.md 없이 Claude Code에게 코드 작성을 요청하는 화면]

Claude가 코드를 작성할 거예요. 결과를 잘 살펴보세요. 어떤 스타일로 코드를 작성했나요? 변수명은 영어인가요 한글인가요? 주석은 있나요? 타입 힌트는 사용했나요?

이 결과를 메모해두세요. 잠시 후 CLAUDE.md를 작성한 다음에 같은 요청을 다시 해볼 거예요. 차이가 확실하게 보일 거예요.

직접 해보기

Claude가 생성한 코드에서 다음 항목을 확인하고 메모해보세요.

- 코딩 스타일: PEP 8을 따르나요?
- 주석 언어: 영어인가요 한글인가요?
- 타입 힌트: 사용하나요?
- 에러 처리: try-except가 있나요?
- 테스트: 테스트 코드도 함께 작성했나요?

이 메모는 Step 4에서 비교할 때 사용할 거예요.

`/exit`으로 Claude Code를 종료하세요. 이제 CLAUDE.md를 작성할 차례예요.

3. CLAUDE.md 작성

3 CLAUDE.md를 직접 작성해요

CLAUDE.md는 프로젝트 루트에 위치하는 마크다운 파일이에요. Claude Code가 프로젝트를 열 때 이 파일을 자동으로 읽어요. 별도의 설정이나 등록 과정이 필요 없어요. 파일이 있으면 읽고, 없으면 안 읽어요. 그만큼 단순해요.

텍스트 에디터로 `CLAUDE.md` 파일을 만들어보세요.

```
code ~/practice-todo-app/CLAUDE.md
```

CLAUDE.md의 구조

CLAUDE.md에는 크게 세 가지를 적어요. 빌드/실행 명령어, 코드 스타일 규칙, 프로젝트 구조예요.

1) 빌드 명령어

프로젝트를 어떻게 실행하고, 테스트하고, 빌드하는지를 적어요. Claude가 코드를 수정한 후 테스트를 돌려보거나, 빌드가 깨지지 않았는지 확인할 때 이 정보를 사용해요.

아래 내용을 CLAUDE.md에 작성해보세요.

```
# Practice Todo App

## 빌드 & 실행
- 실행: `python src/main.py`
- 테스트: `pytest tests/ -v`
- 린트: `ruff check src/`
- 포맷: `ruff format src/`
```

> 빌드 명령어를 적어두면 Claude가 코드를 수정한 후 자동으로 테스트를 실행해요. “테스트 돌려봐”라고 별도로 말하지 않아도 되는 거예요.

2) 코드 스타일 규칙

여러분이 원하는 코딩 스타일을 구체적으로 적어요. 모호하게 적으면 Claude도 모호하게 따르고, 구체적으로 적으면 정확하게 따라요.

CLAUDE.md에 이어서 작성해보세요.

```
## 코드 스타일
- 언어: Python 3.11+
- 타입 힌트: 모든 함수에 필수
- Docstring: Google 스타일, 한글로 작성
- 변수명: snake_case, 의미가 명확한 영어
- 주석: 한글로 작성, "왜"를 설명 (무엇을 하는지는 코드로)
- 에러 처리: 구체적 예외 타입 사용 (bare except 금지)
- import 순서: 표준 라이브러리 → 서드파티 → 로컬 (빈 줄로 구분)
```

3) 프로젝트 구조

프로젝트의 폴더 구조와 각 파일의 역할을 적어요. Claude가 “어디에 코드를 넣어야 하지?”라고 고민하지 않게 해주는 거예요.

```
## 프로젝트 구조
practice-todo-app/
+-- src/
|   +-- main.py          # 엔트리포인트, CLI 인터페이스
|   +-- models.py         # 데이터 모델 (Todo, TodoList)
|   +-- utils.py          # 유틸리티 함수
+-- tests/
|   +-- test_main.py      # 테스트
+-- docs/                  # 문서
+-- CLAUDE.md              # Claude Code 설정
+-- README.md              # 프로젝트 소개

## 규칙
- 새 기능 추가 시 반드시 테스트도 함께 작성
```

```
- 한 함수는 30줄을 넘기지 않음
- 전역 변수 사용 금지
```

[스크린샷: VS Code에서 CLAUDE.md 파일을 작성하고 있는 화면]

전체 CLAUDE.md 완성본

지금까지 작성한 내용을 합치면 아래와 같아요. 전체를 한번에 복사해서 사용해도 괜찮아요.

```
# Practice Todo App

## 빌드 & 실행
- 실행: `python src/main.py`
- 테스트: `pytest tests/ -v`
- 린트: `ruff check src/`
- 포맷: `ruff format src/`

## 코드 스타일
- 언어: Python 3.11+
- 타입 힌트: 모든 함수에 필수
- Docstring: Google 스타일, 한글로 작성
- 변수명: snake_case, 의미가 명확한 영어
- 주석: 한글로 작성, "왜"를 설명 (무엇을 하는지는 코드로)
- 에러 처리: 구체적 예외 타입 사용 (bare except 금지)
- import 순서: 표준 라이브러리 → 서드파티 → 로컬 (빈 줄로 구분)

## 프로젝트 구조
practice-todo-app/
+-- src/
|   +-- main.py          # 엔트리포인트, CLI 인터페이스
|   +-- models.py         # 데이터 모델 (Todo, TodoList)
|   +-- utils.py          # 유틸리티 함수
+-- tests/
|   +-- test_main.py      # 테스트
+-- docs/                  # 문서
+-- CLAUDE.md              # Claude Code 설정
+-- README.md              # 프로젝트 소개

## 규칙
- 새 기능 추가 시 반드시 테스트도 함께 작성
- 한 함수는 30줄을 넘기지 않음
- 전역 변수 사용 금지
```

> CLAUDE.md는 "완벽해야 한다"는 부담을 가질 필요가 없어요. 처음에는 간단하게 시작하고, 프로젝트가 진행되면서 점점 내용을 추가하면 돼요. Claude와 협업하면서 "이것도 적어둬야겠다"는 생각이 들 때마다 업데이트하면 되는 거예요.

CLAUDE.md 작성 팁

CLAUDE.md를 효과적으로 작성하기 위한 팁 몇 가지를 정리해볼게요.

원칙	설명
구체적으로	"좋은 코드를 작성해" 대신 "타입 힌트 필수, Google 스타일 docstring"처럼 구체적으로 적어요
짧게	CLAUDE.md가 너무 길면 오히려 효과가 떨어져요. 핵심만 간결하게 적어요
실행 가능하게	빌드 명령어는 복사해서 바로 실행할 수 있는 형태로 적어요
점진적으로	처음부터 완벽하게 적을 필요 없어요. 필요할 때마다 추가하면 돼요
예시 포함	코드 스타일은 좋은 예시와 나쁜 예시를 함께 적으면 효과적이에요

4. CLAUDE.md 유무로 결과 비교

4 **CLAUDE.md의 효과를 직접 눈으로 확인해요**

이제 CLAUDE.md가 있는 상태에서 같은 요청을 다시 해볼 거예요. Step 2에서 메모해 둔 결과와 비교해보세요.

프로젝트 폴더에서 Claude Code를 다시 실행하세요.

```
cd ~/practice-todo-app
claude
```

먼저 기존 `src/main.py`의 내용을 지워볼게요.

```
> src/main.py 내용을 전부 지우고, 할 일 목록을 관리하는 기본 코드를 새로 작성해줘
```

[스크린샷: CLAUDE.md가 있는 상태에서 같은 요청을 하고 결과를 받는 화면]

결과가 확연히 달라졌을 거예요. Step 2에서의 결과와 비교해보세요.

항목	CLAUDE.md 없을 때	CLAUDE.md 있을 때
타입 힌트	있을 수도, 없을 수도	모든 함수에 타입 힌트 포함
Docstring	영어 or 없음	Google 스타일, 한글
주석 언어	영어	한글
에러 처리	기본적 수준	구체적 예외 타입 사용
테스트	별도 요청 필요	함께 작성됨

직접 해보기

비교를 더 확실하게 해보고 싶다면 아래 실험도 해보세요.

1. CLAUDE.md의 주석 언어 규칙을 "영어로 작성"으로 바꾸고, 같은 요청을 해보세요
2. CLAUDE.md에 "모든 함수 앞에 사용 예시를 docstring에 포함"이라는 규칙을 추가하고, 새 함수를 만들어달라고 해보세요
3. CLAUDE.md에 "출력은 항상 JSON 형태로"라는 규칙을 추가하고, 데이터 모델을 만들어 달라고 해보세요

규칙 하나를 바꿀 때마다 Claude의 출력이 어떻게 달라지는지 관찰해보세요.

CLAUDE.md의 계층 구조

CLAUDE.md는 한 곳에만 있는 것이 아니에요. 세 가지 위치에 둘 수 있고, 각각 다른 범위로 작동해요.

위치	범위	용도
`~/.claude/CLAUDE.md`	모든 프로젝트	개인 선호 (언어, 문체 등)
프로젝트 루트 `CLAUDE.md`	해당 프로젝트	빌드, 스타일, 구조 (팀 공유)
하위 폴더 `CLAUDE.md`	해당 폴더 하위	모듈별 특수 규칙

여러 위치에 CLAUDE.md가 있으면 모두 읽어요. 충돌하는 규칙이 있으면 더 가까운(하위) 파일이 우선해요. 이 구조를 잘 활용하면 팀 전체의 규칙은 루트에, 특정 모듈의 규칙은 하위 폴더에 분리할 수 있어요.

직접 해보기

홈 디렉토리에 개인용 CLAUDE.md를 만들어보세요.

```
mkdir -p ~/.claude
```

`~/.claude/CLAUDE.md`에 아래 내용을 적어보세요.

```
# 개인 설정
- 응답 언어: 한국어
- 코드 주석: 한글
- 설명은 간결하게, 코드는 상세하게
```

이후 어떤 프로젝트에서 Claude Code를 실행하든 이 설정이 기본으로 적용돼요.

자주 쓰는 CLAUDE.md 패턴

실무에서 자주 사용되는 CLAUDE.md 패턴을 몇 가지 소개할게요.

프론트엔드 프로젝트

```
## 코드 스타일
- 프레임워크: React 19 + TypeScript
- 스타일: Tailwind CSS (인라인 스타일 금지)
- 상태관리: Zustand
- 컴포넌트: 함수형만 사용, Props 인터페이스 필수
- 파일명: kebab-case (예: user-profile.tsx)
```

백엔드 API 프로젝트

```
## 코드 스타일
- 프레임워크: FastAPI
- DB: PostgreSQL + SQLAlchemy 2.0
- 모든 엔드포인트에 Pydantic 모델 사용
- 에러 응답: RFC 7807 Problem Details 형식
```

데이터 분석 프로젝트

```
## 코드 스타일
- 환경: Jupyter Notebook + pandas 2.x
- 셀 순서: 데이터 로드 → 전처리 → 분석 → 시각화
- 시각화: matplotlib, 한글 폰트 설정 필수
- 변수명: df_는 DataFrame, s_는 Series 접두사
```

> CLAUDE.md는 프로젝트의 성격에 따라 완전히 달라져요. 프론트엔드, 백엔드, 데이터 분석, 각 도메인에서 중요한 규칙이 다르니까요. 위 예시를 참고해서 여러분의 프로젝트에 맞게 커스터마이즈해보세요.

5. 정리 및 다음 단계

이번 실습 핵심 정리

- **CLAUDE.md**는 Claude Code의 행동을 결정하는 프로젝트 설정 파일이에요
- 빌드 명령어, 코드 스타일, 프로젝트 구조 세 가지를 담아요
- CLAUDE.md가 있으면 Claude의 출력이 일관되고, 프로젝트 규칙을 자동으로 따라요
- 세 가지 위치(홈, 프로젝트 루트, 하위 폴더)에서 계층적으로 작동해요

트러블슈팅

Q. claude 명령어가 실행되지 않아요

A. Node.js 18 이상이 설치되어 있는지 확인하세요. `node --version`으로 확인할 수 있어요. npm 전역 설치 경로가 PATH에 포함되어 있는지도 확인하세요. `npm config get prefix`로 경로를 확인할 수 있어요.

Q. CLAUDE.md를 만들었는데 Claude가 무시하는 것 같아요

A. 파일 이름이 정확히 `CLAUDE.md`인지 확인하세요. 대소문자가 중요해요. `claude.md`나 `Claude.md`가 아닌 `CLAUDE.md`여야 해요. 또한 파일이 프로젝트 루트에 있는지 확인하세요.

Q. Claude가 CLAUDE.md의 규칙을 일부만 따라요

A. CLAUDE.md가 너무 길거나 규칙이 모호하면 일부를 놓칠 수 있어요. 규칙을 줄여보고, 가장 중요한 것부터 위에 배치하세요. 구체적인 예시를 함께 적으면 준수율이 올라가요.

Q. 기존 프로젝트에 CLAUDE.md를 추가하고 싶은데 뭘 적어야 할지 모르겠어요

A. Claude Code에게 물어보세요. `이 프로젝트를 분석하고 CLAUDE.md를 만들어줘`라고 요청하면 프로젝트 구조를 파악해서 초안을 작성해줘요. 거기서 수정하면 훨씬 빨라요.

Q. 팀원들과 CLAUDE.md를 공유하고 싶어요

A. CLAUDE.md는 일반 파일이니까 Git에 커밋하면 돼요. 프로젝트 루트의 CLAUDE.md는 팀 공유용, `~/.claude/CLAUDE.md`는 개인 설정용으로 분리하세요.

실습 체크리스트

- ☐ Claude Code를 설치하고 `claude --version`으로 확인했다
- ☐ 빈 프로젝트에서 CLAUDE.md 없이 코드 생성을 요청하고 결과를 메모했다
- ☐ CLAUDE.md를 작성했다 (빌드 명령어 + 코드 스타일 + 프로젝트 구조)
- ☐ CLAUDE.md가 있는 상태에서 같은 요청을 하고 결과를 비교했다

– 여기까지 완료하면 이번 실습은 성공이에요! –

- ☐ **[도전] 개인용 `~/.claude/CLAUDE.md`를 만들어봤다**
- ☐ **[도전] CLAUDE.md 규칙을 바꿔가며 출력 차이를 3회 이상 실험했다**
- ☐ **[도전] 자기 실무 프로젝트에 맞는 CLAUDE.md를 작성해봤다**

>> 다음 차시 예고

실습 8차시: MCP: 외부 도구 연결

Claude가 파일을 읽고, 웹을 검색하고, 데이터베이스에 접근하게 만들어볼 거예요.
MCP(Model Context Protocol)를 설치하고 직접 연결해보는 실습이에요.